

گزارش پروژه

سامانه مدیریت پرونده‌های پلیس

WP-Project — Police Case Management System

درس: برنامه‌نویسی وب

تاریخ تحویل: اسفند ۱۴۰۴

فهرست مطالب

۳	۱	مسئولیت‌ها و کارهای انجام‌شده توسط هر عضو
۳	۲	قراردادهای توسعه
۳	۱.۲	قراردادهای نامگذاری
۳	۲.۲	قالب پیام‌های کامیت
۴	۳.۲	قراردادهای شاخه‌بندی
۴	۳	نحوه مدیریت پروژه
۴	۴	موجودیت‌های کلیدی سامانه
۴	۱.۴	User — کاربر
۴	۲.۴	RoleProfile — پروفایل نقش
۵	۳.۴	Case — پرونده
۵	۴.۴	Complaint — شکایت
۵	۵.۴	ComplaintReview — بررسی شکایت
۵	۶.۴	Evidence — مدرک
۵	۷.۴	Trial و Punishment — محاکمه و مجازات
۵	۸.۴	PaymentTransaction — تراکنش مالی
۶	۹.۴	RewardReport — گزارش پاداش
۶	۱۰.۴	CitizenTip — اطلاعیه‌ی شهروندی
۶	۵	پکیج‌های NPM استفاده‌شده در پروژه
۶	۱.۵	React (v19)
۶	۲.۵	React Router DOM (v7)
۶	۳.۵	Axios (v1)
۶	۴.۵	Zustand (v5)
۷	۵.۵	Vite (v7)
۷	۶.۵	Vitest (v4)
۷	۶	نمونه کدهای تولیدشده توسط هوش مصنوعی
۷	۱.۶	نمونه ۱ — سرویس احراز هویت در فرانت‌اند
۸	۲.۶	نمونه ۲ — منطق سه ضربه در بک‌اند
۸	۳.۶	نمونه ۳ — کامپوننت محافظ مسیر بر اساس نقش

۹	۷	ضعف‌ها و قوت‌های هوش مصنوعی در توسعه‌ی فرانت‌اند
۹	۱.۷	قوت‌ها
۹	۲.۷	ضعف‌ها
۹	۸	ضعف‌ها و قوت‌های هوش مصنوعی در توسعه‌ی بک‌اند
۹	۱.۸	قوت‌ها
۱۰	۲.۸	ضعف‌ها
۱۰	۹	نیازسنجی‌های ابتدایی و نهایی پروژه
۱۰	۱.۹	نیازسنجی ابتدایی
۱۱	۲.۹	نیازسنجی نهایی (تغییرات و اضافات)
۱۱	۳.۹	قوت‌های تصمیمات گرفته‌شده
۱۱	۴.۹	ضعف‌های تصمیمات گرفته‌شده

۱ مسئولیت‌ها و کارهای انجام‌شده توسط هر عضو

نام عضو تیم	وظایف و مسئولیت‌ها
محمد ابراهیمیان	توسعه فرانت اند و بک اند، پیاده سازی بخشی از داشبورد ماژولار
محمد ایمانی	توسعه بک اند و انجام داکرایز پروژه
مهبد خالقی	توسعه فرانت اند و نوشتن گزارش پروژه، پیاده سازی بخشی از داشبورد ماژولار

۲ قراردادهای توسعه

۱.۲ قراردادهای نامگذاری

• بک‌اند (Django/Python):

- نام فایل‌ها و ماژول‌ها: snake_case (مثال: case_views.py)
- نام کلاس‌ها: PascalCase (مثال: ComplaintReview)
- نام متغیرها و توابع: snake_case (مثال: assigned_detective)
- نام ثابت‌ها: UPPER_SNAKE_CASE (مثال: Status.PENDING_CADET)
- اپلیکیشن‌های Django: نام کوتاه و جمع (مثال: users، cases، evidence)

• فرانت‌اند (React/TypeScript):

- نام کامپوننت‌ها: PascalCase (مثال: CaseDetailPage)
- نام hook‌ها: پیشوند use + camelCase (مثال: useAuthStore)
- نام فایل‌های کامپوننت: PascalCase.tsx
- نام فایل‌های ابزاری: camelCase.ts
- نام متغیرها و توابع: camelCase

۲.۲ قالب پیام‌های کامیت

پیام‌های کامیت از قرارداد Conventional Commits پیروی می‌کنند:

```
<role(A/B/C)-task>: <short description>
```

نمونه‌ی پیام‌های کامیت:

roleC-task:Debugging and added a script for testing

۳.۲ قراردادهای شاخه‌بندی

کاربرد	الگو
شاخه‌ی تولید (production)	main
شاخه‌ی توسعه	dev
قابلیت جدید	feature/<name>
رفع اشکال	fix/<name>
بازآرایی	refactor/<name>

۳ نحوه مدیریت پروژه

در پروژه اکثراً تسک‌ها به صورت مشارکتی بین اعضای تیم بر اساس علاقه‌مندی‌ها و توانمندی‌ها و با همفکری هم در گروه تلگرامی تقسیم می‌شدند که این موضوع به پیشبرد هر چه بهتر پروژه کمک شایانی کرد.

۴ موجودیت‌های کلیدی سامانه

در این بخش موجودیت‌های اصلی سامانه به همراه دلیل وجود آن‌ها توضیح داده می‌شود.

۱.۴ User — کاربر

دلیل وجود: تمام افراد سامانه (شهروند، کادت، افسر، کارآگاه، سروان، کاپیتان، رئیس، پزشک قانونی، قاضی) از یک مدل کاربری واحد مشتق می‌شوند. این موجودیت AbstractUser جنگو را گسترش می‌دهد و فیلدهای اضافی‌ای مانند national_id، phone_number و email را اجباری می‌کند. مدیریت نقش‌ها از طریق سیستم Group جنگو (RBAC پویا) انجام می‌شود که افزودن و حذف نقش را بدون تغییر مدل ممکن می‌سازد.

۲.۴ RoleProfile — پروفایل نقش

دلیل وجود: هر Group جنگو نماینده‌ی یک نقش است. RoleProfile تنها یک فیلد توضیحات به آن اضافه می‌کند تا تعریف نقش‌ها در پنل مدیریت مستند باشد.

۳.۴ Case — پرونده

دلیل وجود: محور اصلی سامانه است. هر جریان کاری (تحقیق، دستگیری، صدور حکم) به یک پرونده متصل است. این موجودیت یک ماشین وضعیت (State Machine) با ۱۲ حالت مشخص دارد (Draft تا Void/Closed) که انتقال‌های غیرمجاز را پیشگیری می‌کند. همچنین سطح اهمیت پرونده (Severity) تعیین می‌کند آیا کیس نیاز به تأیید رئیس پلیس دارد یا خیر.

۴.۴ Complaint — شکایت

دلیل وجود: اولین نقطه‌ی ورود اطلاعات از جانب شهروند است. منطق سه ضربه (three-strike): اگر یک شکایت سه بار توسط بازبین برگردانده شود، به‌صورت خودکار باطل (Void) می‌شود. این قانون مستقیماً در متد register_invalid_attempt پیاده‌سازی شده است.

۵.۴ ComplaintReview — بررسی شکایت

دلیل وجود: فرایند تأیید شکایت دو مرحله دارد: بررسی کادت و بررسی افسر. این موجودیت تاریخچه‌ی تمام تصمیم‌ها را نگه می‌دارد و رد پای مسئولیت (audit trail) ایجاد می‌کند.

۶.۴ Evidence — مدرک

دلیل وجود: مدرک‌ها انواع متفاوتی دارند (رونویسی، پزشکی/بیولوژیک، خودرو، مدرک هویتی). طراحی با موجودیت پایه (Evidence) و زیرموجودیت‌های تخصصی (TranscriptionEvidence, VehicleEvidence, BioMedicalEvidence, IdentityDocumentEvidence) از الگوی Multi-table Inheritance استفاده می‌کند که افزودن انواع جدید را بدون تغییر جدول پایه ممکن می‌سازد.

۷.۴ Punishment و Trial — محاکمه و مجازات

دلیل وجود: پس از اتمام مرحله‌ی تحقیق، پرونده به دادگاه ارسال می‌شود. Trial رأی قاضی را ثبت می‌کند و Punishment (رابطه‌ی یک‌به‌یک با محاکمه) جزئیات مجازات را نگه می‌دارد. جداسازی این دو موجودیت به این دلیل است که امکان دارد محاکمه‌ای بدون صدور مجازات (حکم بی‌گناهی) به پایان برسد.

۸.۴ PaymentTransaction — تراکنش مالی

دلیل وجود: سه نوع پرداخت در سامانه وجود دارد: وثیقه (Bail)، جریمه (Fine) و پاداش (Reward). یک موجودیت واحد با فیلد transaction_type به‌جای سه جدول جداگانه، گزارش‌گیری مالی یکپارچه را ساده‌تر می‌کند.

۹.۴ RewardReport — گزارش پاداش

دلیل وجود: شهروندان می‌توانند اطلاعاتی درباره‌ی مظنونین ارائه دهند و پس از تأیید افسر و کارآگاه، پاداش دریافت کنند. مقدار پاداش بر اساس امتیاز رتبه‌بندی مظنون ($\text{ranking_score} \times 20,000,000$) محاسبه می‌شود.

۱۰.۴ CitizenTip — اطلاعیه‌ی شهروندی

دلیل وجود: جریان مستقل برای دریافت سرنخ از شهروندان در مورد مظنونین تحت تعقیب است. کد رهگیری یکتا (`unique_tracking_code`) پس از مفید شناخته شدن اطلاعیه صادر می‌شود تا شهروند بتواند وضعیت پاداش خود را پیگیری کند.

۵ پکیج‌های NPM استفاده‌شده در پروژه

۱.۵ React (v19)

کارکرد: کتابخانه‌ی اصلی ساخت رابط کاربری مبتنی بر کامپوننت.
توجه: اکوسیستم بزرگ، رندر اعلانی (`declarative rendering`)، پشتیبانی قوی از TypeScript، و تناسب کامل با معماری تک‌صفحه‌ای (SPA) که این پروژه نیاز دارد.

۲.۵ React Router DOM (v7)

کارکرد: مسیریابی سمت کلاینت؛ تعریف مسیرها، پارامترهای پویا، و ریدایرکت‌ها.
توجه: با داشتن بیش از ۱۰ نقش کاربری و صفحات متفاوت برای هر نقش، نیاز به یک سیستم مسیریابی اعلانی قوی اجتناب‌ناپذیر بود. نسخه ۷ قابلیت `lazy loading` را نیز بهتر پشتیبانی می‌کند.

۳.۵ Axios (v1)

کارکرد: کلاینت HTTP برای ارتباط با REST API بک‌اند.
توجه: در مقایسه با `fetch` بومی، `Axios` interceptorهای خودکار توکن JWT و خطاهای 401 را بسیار ساده‌تر می‌کنند.

۴.۵ Zustand (v5)

کارکرد: مدیریت وضعیت سراسری سبک‌وزن.
توجه: وضعیت احراز هویت و اطلاعات کاربر جاری باید در تمام کامپوننت‌ها در دسترس باشند. Zustand بدون boilerplate زیاد (در مقایسه با Redux) این کار را انجام می‌دهد و با TypeScript نیز به‌خوبی کار می‌کند.

۵.۵ Vite (v7)

کارکرد: ابزار build و سرور توسعه.

توجه: سرعت بالای Hot Module Replacement، زمان راه‌اندازی فوری، و پیکربندی ساده آن را به انتخاب پیش‌فرض اکثر پروژه‌های جدید React تبدیل کرده است.

۶.۵ Vitest (v4)

کارکرد: فریم‌ورک تست یکپارچه با Vite.

توجه: چون پروژه از Vite استفاده می‌کند، Vitest بدون تنظیمات اضافه در همان محیط build اجرا می‌شود و سرعت اجرای تست‌ها را به‌طور قابل‌توجهی بالا می‌برد.

۶ نمونه کدهای تولیدشده توسط هوش مصنوعی

۱.۶ نمونه ۱ — سرویس احراز هویت در فرانت‌اند

کد زیر توسط هوش مصنوعی تولید شده است و مدیریت توکن JWT با Axios interceptor را نشان می‌دهد:

Listing : auth interceptor (TypeScript)

```
// src/services/api.ts
import axios from 'axios';
import { useAuthStore } from '../store/authStore';

const api = axios.create({ baseURL: '/api' });

api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401) {
      await useAuthStore.getState().refresh();
      return api(error.config);
    }
    return Promise.reject(error);
  }
);
```



```

    },
);

export default api;

```

۲.۶ نمونه ۲ — منطق سه ضربه در بک‌اند

کد زیر توسط هوش مصنوعی پیشنهاد شد و منطق باطل‌شدن خودکار شکایت را پیاده‌سازی می‌کند:

Listing : Complaint three-strike logic (Python)

```

# cases/models.py
def register_invalid_attempt(self):
    self.invalid_attempt_count += 1
    if self.invalid_attempt_count >= 3:
        self.status = self.Status.VOID
    else:
        self.status = self.Status.RETURNED
    self.save(update_fields=[
        "invalid_attempt_count", "status", "updated_at"
    ])

```

۳.۶ نمونه ۳ — کامپوننت محافظ مسیر بر اساس نقش

Listing : RoleGuard component (TypeScript/React)

```

// src/components/RoleGuard.tsx
import { Navigate } from 'react-router-dom';
import { useAuthStore } from '../store/authStore';

interface Props {
    allowedRoles: string[];
    children: React.ReactNode;
}

export function RoleGuard({ allowedRoles, children }: Props) {
    const { user } = useAuthStore();

    if (!user) return <Navigate to="/login" replace />;

```

```
const hasRole = user.roles.some((r) => allowedRoles.includes(r));
if (!hasRole) return <Navigate to="/403" replace />;

return <>{children}</>;
}
```

۷ ضعف‌ها و قوت‌های هوش مصنوعی در توسعه‌ی فرانت‌اند

۱.۷ قوت‌ها

- تولید کامپوننت‌های تکراری: برای تولید فرم‌ها، جداول و صفحات مشابه برای ۱۰ نقش کاربری مختلف، هوش مصنوعی به‌طور چشمگیری زمان کدنویسی را کاهش داد.
- پیشنهاد الگوهای رابط کاربری: ساختارهای رابط کاربری مانند RoleGuard، lazy loading، مسیرها، و مدیریت وضعیت بارگذاری و خطا به‌خوبی پیشنهاد شدند.
- رفع خطاهای تایپ TypeScript: تشخیص خطاهای نوع‌دهی و پیشنهاد نوع‌های صحیح برای props و API response‌ها دقیق بود.
- نوشتن تست‌های واحد: تولید تست‌های Vitest برای کامپوننت‌های ساده سریع و کاربردی بود.

۲.۷ ضعف‌ها

- درک ضعیف از منطق تجاری پیچیده: صفحات مرتبط با جریان‌های چند مرحله‌ای (مانند تابلوی کارآگاه) نیاز به تصحیح‌های متعدد داشتند زیرا هوش مصنوعی زمینه‌ی کامل جریان کاری را نمی‌دانست.
- ناسازگاری با ساختار API واقعی: کدهای تولیدشده اغلب فرض‌های اشتباهی درباره‌ی ساختار پاسخ API داشتند که باید دستی اصلاح می‌شدند.
- پیشنهاد کتابخانه‌های اضافی: هوش مصنوعی گاهی پکیج‌های جدید (مانند react-query) پیشنهاد می‌داد که با معماری انتخاب‌شده تعارض داشت.

۸ ضعف‌ها و قوت‌های هوش مصنوعی در توسعه‌ی بک‌اند

۱.۸ قوت‌ها

- طراحی مدل‌ها: طراحی اولیه‌ی مدل‌های Django (از جمله Multi-table Inheritance برای مدرک‌ها) با کیفیت بالایی توسط هوش مصنوعی پیشنهاد شد.

- نوشتن Serializer ها و ViewSet ها: تولید serializer های تودرتو و permission class های سفارشی دقیق بود.
- پیاده‌سازی State Machine: منطق ماشین وضعیت پرونده با جدول انتقال‌های صریح به‌درستی تولید شد.
- نوشتن تست‌های یکپارچگی: تست‌های Django REST Framework با APIClient برای جریان‌های کامل به‌خوبی تولید شدند.

۲.۸ ضعف‌ها

- خطاهای مربوط به migration: پس از تغییرات مدل، هوش مصنوعی گاهی migration های ناسازگار پیشنهاد می‌داد که باید دستی بررسی می‌شدند.
- درک ناقص از RBAC پویا: منطق بررسی مجوزها با Group - محور بودن سیستم گاهی اشتباه پیاده‌سازی می‌شد.
- امنیت: در برخی موارد هوش مصنوعی بررسی مالکیت شیء را فراموش می‌کرد (مثلاً بررسی اینکه آیا کاربر جاری مجاز به ویرایش یک مدرک خاص است).

۹ نیازسنجی‌های ابتدایی و نهایی پروژه

۱.۹ نیازسنجی ابتدایی

در ابتدای پروژه، نیازهای اصلی به صورت زیر تعریف شدند:

۱. سامانه‌ی ثبت و مدیریت شکایت شهروندان
۲. جریان تأیید چند مرحله‌ای (کادت → افسر)
۳. مدیریت پرونده‌ها با نقش‌های مختلف پلیسی
۴. زیرسامانه‌ی مدیریت مدارک و شواهد
۵. زیرسامانه‌ی مالی (وثیقه، جریمه، پاداش)
۶. واسط کاربری مبتنی بر نقش

۲.۹ نیازسنجی نهایی (تغییرات و اضافات)

در طول توسعه، نیازهای زیر به پروژه اضافه یا اصلاح شدند:

۱. تأیید دوگانه برای پرونده‌های بحرانی: پرونده‌های با سطح بحرانی (Critical) نیاز به تأیید رئیس پلیس (Chief) علاوه بر کاپیتان دارند.
۲. سامانه‌ی مزنونین تحت تعقیب: رتبه‌بندی مزنونین و کانال اطلاعیه‌ی شهروندی در نیازسنجی اولیه وجود نداشت.
۳. تابلوی گرافی کارآگاه: نیاز به ابزار بصری برای مدیریت ارتباط مدارک و مزنونین در طول توسعه مشخص شد.
۴. زیرسامانه‌ی پزشکی قانونی: گردش کار تأیید مدارک پزشکی توسط پزشک قانونی دیرتر اضافه شد.

۳.۹ قوت‌های تصمیمات گرفته‌شده

- استفاده از RBAC پویا با Group جنگو: افزودن نقش‌های جدید بدون تغییر کد ممکن شد.
- طراحی State Machine صریح برای پرونده: جلوگیری از انتقال‌های غیرمجاز به‌صورت خودکار.
- جداسازی apps بر اساس domain: انسجام بالای کد و آزمون‌پذیری بهتر.

۴.۹ ضعف‌های تصمیمات گرفته‌شده

- یکپارچگی درگاه پرداخت: در نیازسنجی ابتدایی یکپارچگی واقعی با درگاه در نظر گرفته نشد و همچنان ناقص است.
- پیچیدگی مدل Case: متمرکز کردن تمام وضعیت‌ها در یک مدل واحد منجر به رشد ماشین وضعیت شد و نگهداری آن را سخت‌تر کرد.